Here’s a concrete folder layout plus a JWT auth design doc you can almost drop into your repo.

Folder structure

text

```
library-management/
 ├── app/
|   ├── main.py
|   |   # FastAPI app creation, router includes, startup/shutdown
|   |
|   ├── core/
|   |   ├── config.py
|   |   |   # Settings (DB URL, JWT secret, algorithm, token expiry)
|   |   ├── security.py
|   |       # Password hashing/verify
|   |       # JWT access token creation & decoding
|   |
|   ├── db/
|   |   ├── database.py
|   |   |   # SQLAlchemy engine, SessionLocal, Base
|   |   ├── models.py
|   |       # Users, Books, IssuedBooks, Waitlist, BookRequests models
|   |
|   ├── schemas/
|   |   ├── user.py
|   |   |   # UserCreate, UserLogin, UserOut, Token, TokenData
|   |   ├── book.py
|   |   |   # BookCreate, BookUpdate, BookOut
|   |   ├── issue.py
|   |       # IssueCreate, IssueOut, ReturnPayload, FineOut
|   |
|   ├── crud/
```

```
|   |   ├── user.py
|   |   |   # get_user_by_email, create_user, get_user, list_students
|   |   ├── book.py
|   |   |   # CRUD for books, stock updates
|   |   ├── issue.py
|   |       # CRUD for issued_books plus queries (active, overdue)
|   |
|   ├── api/
|   |   ├── deps.py
|   |   |   # get_db()
|   |   |   # get_current_user()
|   |   |   # get_current_librarian()
|   |   |   # get_current_student()
|   |   ├── routes/
|   |       ├── auth.py      # /auth/login
|   |       ├── users.py     # librarian-only user ops
|   |       ├── books.py     # list/create/update books
|   |       ├── issues.py    # issue/return endpoints
|   |       ├── waitlist.py  # join/cancel/list waitlist
|   |       ├── requests.py  # book_requests
|   |
|   ├── services/
|   |   ├── issue_service.py
|   |   |   # transactional issue/return, fine calculation
|   |   ├── waitlist_service.py
|   |       # queue maintenance, allocation logic
|   |
|   └── utils/
|       ├── constants.py     # roles, status enums, fine rate, loan period
|
```

```
├── requirements.txt
├── .env                # DB URL, JWT_SECRET, JWT_ALGO, ACCESS_TOKEN_EXPIRE_MINUTES
└── README.md
```

Key separation:

- **api/**: HTTP only (request/response wiring, dependencies).
- **crud/**: raw DB access (no business rules).
- **services/**: real business logic (issue/return, fine, waitlist).
- **core/**: configuration + security (JWT, hashing).
  This keeps JWT and RBAC centralised instead of scattered across routes.

JWT auth design

1. Core concepts

- Auth is **stateless**: no sessions in DB; only JWT access tokens.
- Token type: **access token only**, short-lived (e.g., 30–60 minutes), no refresh token for now.
- Token contents:
  - sub: user id (string)
  - role: librarian or student
  - exp: expiry timestamp

All protected routes will:

1. Read Authorization: Bearer <token>.
2. Decode and validate token.
3. Load user from DB.
4. Enforce role in dependency (not in route body).

2. Where each piece lives

2.1 core/config.py

Responsibilities:

- Load .env and central settings.
- Expose:
  - DATABASE_URL
  - JWT_SECRET_KEY
  - JWT_ALGORITHM (e.g. HS256)
  - ACCESS_TOKEN_EXPIRE_MINUTES

    - Any other constants that shouldn't be hardcoded.

Example fields (conceptual):

- JWT_SECRET_KEY: strong random string.
- ACCESS_TOKEN_EXPIRE_MINUTES: 30.

2.2 core/security.py

Responsibilities:

- Password hashing and verification.
- JWT token creation.
- JWT token decoding/validation (used by dependencies).

Logical responsibilities:

1. **Password hashing**
    - Use a strong hash (e.g., bcrypt via passlib).
    - Store only hashes in users.password_hash.
    - Functions:
        - get_password_hash(plain_password) -> str
        - verify_password(plain_password, hashed_password) -> bool
2. **Access token creation**
    - Function:
        - create_access_token(data: dict, expires_delta: timedelta | None) -> str
    - Steps:
        - Copy payload data.
        - Add exp = UTC now + expiry.
        - Encode with JWT_SECRET_KEY and JWT_ALGORITHM.

Typical payload for login:

```
json
{
  "sub": "user_id_as_string",
  "role": "librarian"
}
```

3. **Token decoding**
    - Function:

        - decode_access_token(token: str) -> dict
    - Steps:
        - Decode using same secret and algo.
        - Catch signature/expiry errors.
        - Ensure sub and role are present.
    - Return minimal identity info (e.g. user_id, role) which is used by get_current_user.

2.3 schemas/user.py

Key Pydantic models:

- UserLogin:
    - email, password.
- UserOut:
    - id, name, email, role.
- Token:
    - access_token, token_type (always "bearer").
- TokenData:
    - user_id (optional), role (optional).

These models ensure the auth endpoints and dependencies have typed payloads.

2.4 api/routes/auth.py

Endpoint: POST /auth/login.

Flow:

1. Accept UserLogin payload.
2. Use crud.user.get_user_by_email to load user.
3. If not found or password invalid → authentication error.
4. Create token with:
    - sub = str(user.id)
    - role = user.role
5. Return:

```
json
{
  "access_token": "<jwt_here>",
  "token_type": "bearer"
```

}

This file must not:

- Hash passwords directly (delegates to core/security.py).
- Do DB logic directly (delegates to crud/user.py).

2.5 api/deps.py

This is the **RBAC backbone**.

Typical dependencies:

1. get_db()
    - Yields SQLAlchemy session.
2. get_current_user()
    - Reads Authorization: Bearer <token>.
    - Validates token using core.security.decode_access_token.
    - Extracts user_id and role.
    - Loads user from DB via crud.user.get.
    - Raises 401 if invalid/missing/expired or user inactive.
    - Returns user model.
3. get_current_librarian()
    - Calls get_current_user().
    - Checks user.role == "librarian".
    - Raises 403 if not.
    - Returns user.
4. get_current_student()
    - Similar to above, but enforces role == "student".

Usage in routes:

- Librarian-only:
    - current_user: User = Depends(get_current_librarian)
- Student-only:
    - current_user: User = Depends(get_current_student)
- Any authenticated:
    - current_user: User = Depends(get_current_user)

This keeps role checks **outside** route logic.

2.6 How routes use auth

Examples (conceptual, not code):

- books.py
    - GET /books:
        - Depends(get_current_user) → both roles can view.
    - POST /books (add book):
        - Depends(get_current_librarian) → only librarian.
- issues.py
    - POST /issues/issue:
        - Depends(get_current_librarian) → only librarian issues.
    - POST /issues/{issue_id}/return:
        - Depends(get_current_librarian).
- waitlist.py
    - POST /books/{book_id}/waitlist:
        - Depends(get_current_student) → only students join queue.
    - GET /books/{book_id}/waitlist:
        - Depends(get_current_librarian) → librarian sees full queue.

This matches your rule matrix:

- Add book → librarian.
- Issue book → librarian.
- Join waitlist → student.
- View books → both.

3. JWT auth flow (mapped to files)

1. **Login request**
    - Route: POST /auth/login in api/routes/auth.py.
    - Body: { "email": "...", "password": "..." }.
2. **Credential check**
    - crud/user.py:
        - get_user_by_email(email).
    - core/security.py:
        - verify_password.

3. **Token creation**
    - core/security.py:
        - create_access_token({ "sub": user.id, "role": user.role }).
4. **Response**
    - Token schema:

```
json
{
  "access_token": "...",
  "token_type": "bearer"
}
```

5. **Authenticated requests**
    - Client sets Authorization: Bearer <token>.
6. **Request handling**
    - api/deps.py:
        - get_current_user:
            - Extract and decode token.
            - Load user.
        - get_current_librarian / get_current_student:
            - Enforce role.
7. **Business logic**
    - services/issue_service.py:
        - Uses current_librarian + DB session to:
            - Issue books (check availability, decrement count, create issued_books row).
            - Return books (fine, increment count, process waitlist).

4. Security rules & pitfalls

Enforce these explicitly:

- **No plain passwords**:
    - Always hash on registration/seed.
    - Always verify using verify_password.
- **Token expiry**:

    - Always include exp.
    - Reject expired tokens in decoder.
- **RBAC everywhere**:
    - Never rely on the client to say it's a librarian.
    - Always use dependencies for role checks.
- **No business logic in routes**:
    - Routes should delegate to:
        - crud for DB.
        - services for complex flows.
- **Single source of JWT logic**:
    - All token encode/decode must go through core/security.py, not duplicated.

If you want, the next step can be a short "JWT Auth Spec.md" with bulletproof sections you can submit (overview, claims design, dependencies, error scenarios, and example request–response).